Deep Merge Utility Library

A Powerful TypeScript Library for Advanced Object and Array Merging

---

Table of Contents

1. Overview
2. Features
3. Installation
4. Core Functions
   - mergeArrayValues
   - deepMerge
   - deepMergeWithDefault
   - deepMergeAll
   - deepMergeAllExtended
5. Merge Strategies
6. Usage Examples
7. API Reference
8. Type Definitions
9. Best Practices
10. Contributing
11. License

Overview

The **Deep Merge Utility Library** provides robust, type-safe solutions for merging complex JavaScript objects and arrays in TypeScript applications. Whether you're working with configuration files, API responses, user preferences, or nested data structures, this library offers flexible and powerful merging capabilities with full TypeScript support.

Use Cases

- **Configuration Management**: Merge default settings with user configurations

- **API Response Handling**: Combine multiple API responses intelligently
- **State Management**: Merge application state across different modules
- **Settings Aggregation**: Layer multiple configuration sources (system, user, admin)
- **Data Synchronization**: Merge datasets from different sources

---

Features

Feature	Status	Description
Recursive Merging	✅	Deep merge nested objects to any depth
TypeScript Inference	✅	Full type safety with automatic type inference
Multiple Object Merging	✅	Merge two or more objects in a single operation
Custom Array Strategies	✅	Define how arrays should be merged (concat, replace, unique, custom)
Built-in Type Support	✅	Native support for `Map`, `Set`, and `Date` objects
Flexible Return Types	✅	Automatically inferred intersection types for merged results
Zero Dependencies	✅	Lightweight library with no external dependencies

---

Installation

```bash
npm install deep-merge-utils
# or
yarn add deep-merge-utils
# or
pnpm add deep-merge-utils
```

Basic Import

```typescript
```

```typescript
import {
  mergeArrayValues,
  deepMerge,
  deepMergeWithDefault,
  deepMergeAll,
  deepMergeAllExtended,
  type MergeArrayStrategy,
  type MergeObjects
} from 'deep-merge-utils';
```

Core Functions

mergeArrayValues

Merges two arrays according to a specified strategy.

Signature

```typescript
function mergeArrayValues<T>(
  target: T[],
  source: T[],
  strategy: MergeArrayStrategy<T> = "mergeUnique"
): T[]
```

Parameters

Parameter	Type	Default	Description
`target`	`T[]`	-	The base/destination array
`source`	`T[]`	-	The array to merge with the target
`strategy`	`MergeArrayStrategy<T>`	`"mergeUnique"`	The merging strategy to apply

Return Value

Returns a new merged array based on the specified strategy.

Strategies

- **`'concat'`**: Concatenates both arrays
- **`'replace'`**: Replaces target with source
- **`'mergeUnique'`**: Merges and removes duplicates

- **Custom Function**: Implement your own merging logic

Example

```typescript
const arr1 = [1, 2, 3];
const arr2 = [4, 5, 6];

// Concatenate arrays
const result1 = mergeArrayValues(arr1, arr2, 'concat');
console.log(result1); // [1, 2, 3, 4, 5, 6]

// Replace with source
const result2 = mergeArrayValues(arr1, arr2, 'replace');
console.log(result2); // [4, 5, 6]

// Merge with unique values
const numbers1 = [1, 2, 3];
const numbers2 = [3, 4, 5];
const result3 = mergeArrayValues(numbers1, numbers2, 'mergeUnique');
console.log(result3); // [1, 2, 3, 4, 5]
```

---

deepMerge

Performs a deep recursive merge of two objects with support for special types.

Signature

```typescript
function deepMerge<T extends object, D extends Partial<T>>(
  obj: T | Partial<T>,
  defaults: D,
  mergeArrays: MergeArrayStrategy<any> = 'replace'
): T
```

Parameters

Parameter	Type	Default	Description
`obj`	`T | Partial<T>`	-	The source object to merge
`defaults`	`D`	-	The default/base object
`mergeArrays`	`MergeArrayStrategy<any>`	`'replace'`	Array merging strategy

Features

- Recursively merges nested objects
- Handles special types: `Map`, `Set`, `Date`
- Preserves undefined values from source
- Type-safe with TypeScript generics

Example

```typescript
```

```typescript
type Options = {
  security: {
    csrf: boolean;
    xss: boolean;
  };
  caching: {
    enabled: boolean;
    ttl: number;
  };
};

const userOptions: Partial<Options> = {
  security: {
    csrf: true
  }
};

const defaultOptions: Options = {
  security: {
    csrf: false,
    xss: true
  },
  caching: {
    enabled: true,
    ttl: 300
  }
};

const merged = deepMerge(userOptions, defaultOptions);

console.log(merged);
/* Output:
{
  security: {
    csrf: true,      // User preference overrides default
    xss: true        // Default value preserved
  },
  caching: {
    enabled: true,
    ttl: 300
  }
}
*/
```

deepMergeWithDefault

A convenience wrapper for `deepMerge` that handles undefined source objects.

Signature

```typescript
function deepMergeWithDefault<T extends object>(
  obj: T | undefined,
  defaults: T
): T
```

Parameters

Parameter	Type	Description
`obj`	`T | undefined`	The source object (may be undefined)
`defaults`	`T`	The default/base object

Return Value

Returns the merged object, treating undefined as an empty object.

Example

```typescript
```

```typescript
type Config = {
  apiUrl: string;
  timeout: number;
  retries: number;
};

const defaultConfig: Config = {
  apiUrl: 'https://api.example.com',
  timeout: 5000,
  retries: 3
};

// Handles undefined gracefully
const config1 = deepMergeWithDefault(undefined, defaultConfig);
console.log(config1); // Returns defaultConfig

// Also works with partial objects
const userConfig: Partial<Config> = { timeout: 10000 };
const config2 = deepMergeWithDefault(userConfig, defaultConfig);
console.log(config2);
/* Output:
{
  apiUrl: 'https://api.example.com',
  timeout: 10000,
  retries: 3
}
*/
```

---

deepMergeAll

Merges multiple objects sequentially using a specified array strategy.

Signature

```typescript
function deepMergeAll<T>(
  mergeStrategy: MergeArrayStrategy<T> = 'replace',
  ...objects: Partial<T>[]
): T
```

Parameters

Parameter	Type	Default	Description
mergeStrategy	MergeArrayStrategy<T>	'replace'	Array merging strategy for all objects
...objects	Partial<T>[]	-	Variable number of objects to merge

Return Value

Returns a single merged object combining all provided objects.

Behavior

Objects are merged from left to right, with later objects overriding earlier ones. All arrays are merged according to the specified strategy.

Example

```typescript
```

```typescript
interface AppConfig {
  name: string;
  version: string;
  settings: {
    theme: string;
    notifications: boolean;
  };
  features: string[];
}

const defaultConfig: Partial<AppConfig> = {
  name: 'My App',
  version: '1.0.0',
  settings: {
    theme: 'dark',
    notifications: true
  },
  features: ['dashboard', 'reports']
};

const userConfig: Partial<AppConfig> = {
  settings: {
    theme: 'light'
  },
  features: ['charts', 'export']
};

const adminOverrides: Partial<AppConfig> = {
  version: '1.0.1-beta',
  settings: {
    notifications: false
  },
  features: ['admin-panel']
};

// Merge with 'replace' strategy (default)
const finalConfig = deepMergeAll<AppConfig>(
  'replace',
  defaultConfig,
  userConfig,
  adminOverrides
);

console.log(JSON.stringify(finalConfig, null, 2));
/* Output:
{
```

```
  "name": "My App",
  "version": "1.0.1-beta",
  "settings": {
    "theme": "light",
    "notifications": false
  },
  "features": ["admin-panel"]
}
*/

// Merge with 'concat' strategy
const concatConfig = deepMergeAll<AppConfig>(
  'concat',
  defaultConfig,
  userConfig,
  adminOverrides
);

console.log(JSON.stringify(concatConfig, null, 2));
/* Output:
{
  "name": "My App",
  "version": "1.0.1-beta",
  "settings": {
    "theme": "light",
    "notifications": false
  },
  "features": ["dashboard", "reports", "charts", "export", "admin-panel"]
}
*/
```

---

deepMergeAllExtended

Advanced version of `deepMergeAll` with enhanced type inference for multiple objects.

Signature

```typescript
function deepMergeAllExtended<T extends object[]>(
  mergeArrays: MergeArrayStrategy<T>,
  ...objects: T
): MergeObjects<T>
```

Parameters

Parameter	Type	Description
mergeArrays	MergeArrayStrategy<T>	Array merging strategy
...objects	T	Objects to merge

Return Value

Returns a merged object with automatically inferred intersection type of all input objects.

Type Safety

The return type is the intersection of all provided object types, ensuring complete type safety and IDE autocomplete support.

Example

```typescript
const defaults = {
  env: 'prod',
  security: {
    headers: ['X-Frame-Options']
  },
  cache: new Map([['enabled', true]]),
  lastModified: new Date('2023-01-01')
};

const userConfig = {
  env: 'dev',
  security: {
    headers: ['Content-Security-Policy']
  },
  cache: new Map([['ttl', 300]])
};

const final = deepMergeAllExtended('concat', defaults, userConfig);

/* Inferred type:
{
  env: string;
  security: { headers: string[] };
  cache: Map<string, any>;
  lastModified: Date;
}
*/
```

Merge Strategies

Strategy Overview

Strategy	Behavior	Best For
`'concat'`	Combines all array elements	Aggregating data from multiple sources
`'replace'`	Uses source array, discards target	Overwriting with explicit values
`'mergeUnique'`	Combines and removes duplicates	Maintaining unique values
Custom Function	User-defined logic	Complex merging requirements

Detailed Strategy Descriptions

1. Concat Strategy

```typescript
const result = mergeArrayValues([1, 2], [3, 4], 'concat');
// Result: [1, 2, 3, 4]
```

Use Case: Combining feature lists, logging formats, or any additive scenario.

2. Replace Strategy

```typescript
const result = mergeArrayValues([1, 2], [3, 4], 'replace');
// Result: [3, 4]
```

Use Case: Overwriting entire arrays with explicit configurations.

3. Merge Unique Strategy

```typescript
const result = mergeArrayValues([1, 2, 3], [2, 3, 4], 'mergeUnique');
// Result: [1, 2, 3, 4]
```

Use Case: Maintaining unique elements across configurations.

4. Custom Function Strategy

```typescript
```

```
const customStrategy: MergeArrayStrategy<number> = (target, source) => {
  const evenNumbers = source.filter(n => n % 2 === 0);
  return [...target, ...evenNumbers];
};

const result = mergeArrayValues([1, 2], [3, 4, 5, 6], customStrategy);
// Result: [1, 2, 4, 6]
```

Use Case: Implementing domain-specific merging logic.

Usage Examples

Example 1: Application Configuration Layer

```typescript
```

```
// System configuration
const systemConfig = {
  database: {
    host: 'localhost',
    port: 5432,
    pool: 10
  },
  logging: ['file', 'console'],
  debug: false
};

// Environment configuration
const envConfig = {
  database: {
    host: process.env.DB_HOST || 'localhost'
  },
  debug: process.env.NODE_ENV === 'development'
};

// User overrides
const userConfig = {
  logging: ['file']
};

const finalConfig = deepMergeAll(
  'concat',
  systemConfig,
  envConfig,
  userConfig
);

console.log(finalConfig);
/* Output:
{
  database: {
    host: 'development.db.example.com',
    port: 5432,
    pool: 10
  },
  logging: ['file', 'console', 'file'],
  debug: true
}
*/
```

Example 2: API Response Aggregation

```typescript
```

```typescript
interface UserPreferences {
  theme: string;
  language: string;
  notifications: {
    email: boolean;
    push: boolean;
  };
  features: string[];
}

// Default preferences
const defaults: UserPreferences = {
  theme: 'light',
  language: 'en',
  notifications: {
    email: true,
    push: true
  },
  features: ['basic', 'notifications']
};

// User saved preferences
const userPrefs: Partial<UserPreferences> = {
  theme: 'dark',
  notifications: {
    email: false
  },
  features: ['basic', 'notifications', 'advanced']
};

// Organization defaults
const orgDefaults: Partial<UserPreferences> = {
  language: 'fr',
  features: ['basic', 'team-collaboration']
};

const effectivePreferences = deepMergeAll<UserPreferences>(
  'mergeUnique',
  defaults,
  orgDefaults,
  userPrefs
);

console.log(effectivePreferences);
/* Output:
{
```

```
  theme: 'dark',
  language: 'fr',
  notifications: {
    email: false,
    push: true
  },
  features: ['basic', 'notifications', 'advanced', 'team-collaboration']
}
*/
```

Example 3: Custom Merge Logic

```typescript
// Filter strategy: only accept strings longer than 5 characters
const filterStrategy: MergeArrayStrategy<string> = (target, source) => {
  return [...target, ...source].filter(s => s.length > 5);
};

interface Features {
  enabled: string[];
}

const defaults: Features = {
  enabled: ['authentication', 'logging']
};

const user: Partial<Features> = {
  enabled: ['api', 'advanced-analytics']
};

const result = deepMergeAll<Features>(
  filterStrategy as any,
  defaults,
  user
);

console.log(result);
// Output: { enabled: ['authentication', 'logging', 'advanced-analytics'] }
```

Example 4: Working with Special Types

```typescript
```

```typescript
interface AdvancedConfig {
  cache: Map<string, number>;
  tags: Set<string>;
  created: Date;
}

const config1: Partial<AdvancedConfig> = {
  cache: new Map([['ttl', 300]]),
  tags: new Set(['production']),
  created: new Date('2023-01-01')
};

const config2: Partial<AdvancedConfig> = {
  cache: new Map([['maxSize', 1000]]),
  tags: new Set(['critical']),
  created: new Date('2024-01-01')
};

const merged = deepMerge(config1, config2);

console.log(merged.cache); // Map with entries from both
console.log(merged.tags); // Set with tags from both
console.log(merged.created); // Date from config2
```

API Reference

Type Definitions

MergeArrayStrategy<T>

```typescript
type MergeArrayStrategy<T> =
  | 'concat'
  | 'replace'
  | 'mergeUnique'
  | ((target: T[], source: T[]) => T[]);
```

Defines how arrays should be merged. Can be a predefined strategy string or a custom function.

MergeObjects<T>

```typescript
```

```typescript
type MergeObjects<T extends any[]> =
  T extends [infer First, ...infer Rest]
    ? First & MergeObjects<Rest>
    : unknown;
```

Computes the intersection type of multiple objects for precise type inference.

Function Summary

Function	Purpose	Parameters	Returns
`mergeArrayValues`	Merge two arrays	`target`, `source`, `strategy`	Merged array
`deepMerge`	Deep merge two objects	`obj`, `defaults`, `mergeArrays`	Merged object
`deepMergeWithDefault`	Merge with undefined handling	`obj`, `defaults`	Merged object
`deepMergeAll`	Merge multiple objects	`mergeStrategy`, `...objects`	Merged object
`deepMergeAllExtended`	Advanced multi-merge	`mergeArrays`, `...objects`	Merged object with inferred type

---

Best Practices

1. Choose the Right Strategy

```typescript
// ✗ Wrong: Using 'concat' when you want replacement
const result = deepMergeAll('concat', base, override);

// ✓ Correct: Use 'replace' for overrides
const result = deepMergeAll('replace', base, override);
```

2. Leverage TypeScript Types

```typescript
// ✓ Define explicit interfaces
interface Config {
  api: { url: string; timeout: number };
  features: string[];
}

const config = deepMerge(userConfig, defaults) as Config;
```

3. Handle Edge Cases

```typescript
// ✅ Use deepMergeWithDefault for optional inputs
const config = deepMergeWithDefault(userInput, defaults);

// ✅ Validate special types explicitly
if (result.cache instanceof Map) {
  // Process Map
}
```

4. Use Custom Strategies Wisely

```typescript
// ✅ Keep custom strategies focused and pure
const deduplicateStrategy: MergeArrayStrategy<string> = (t, s) =>
  Array.from(new Set([...t, ...s]));

// ❌ Avoid side effects in strategies
const badStrategy: MergeArrayStrategy<any> = (t, s) => {
  console.log('Merging...'); // Side effect!
  return [...t, ...s];
};
```

5. Order Matters

```typescript
// Objects are merged left to right, later objects override earlier ones
const result = deepMergeAll(
  'replace',
  defaults,       // Applied first
  envConfig,      // Overrides defaults
  userConfig      // Final overrides
);
```

Performance Considerations

- **Shallow vs Deep**: This library performs deep recursion. For large nested objects, consider if a shallow merge is sufficient.
- **Special Types**: Merging `Map`, `Set`, and `Date` objects involves copying. Be mindful with very large collections.

- **Custom Strategies**: Complex custom strategies may impact performance. Test with realistic data sizes.

Troubleshooting

Issue: Type Errors with Merged Objects

Solution: Use explicit type annotations:

```typescript
const result = deepMerge(obj, defaults) as Config;
// or
const result = deepMerge<Config>(obj, defaults);
```

Issue: Unexpected Array Behavior

Solution: Verify your merge strategy:

```typescript
// Check what strategy is being used
console.log('Using strategy: replace'); // or 'concat', 'mergeUnique'
```

Issue: Special Types Not Merging

Solution: Ensure both source and target have the same special type:

```typescript
// ✅ Both are Maps
const map1 = new Map();
const map2 = new Map();
deepMerge({ cache: map1 }, { cache: map2 });

// ❌ Type mismatch - won't merge as special type
deepMerge({ cache: map1 }, { cache: {} });
```

Contributing

We welcome contributions! Please follow these guidelines:

1. Fork the repository
2. Create a feature branch: `git checkout -b feature/your-feature`
3. Commit your changes: `git commit -am 'Add new feature'`

4. Push to the branch: `git push origin feature/your-feature`
5. Submit a pull request

Code Standards

- Maintain TypeScript strict mode
- Add comprehensive test cases
- Include JSDoc comments for public APIs
- Follow existing code style

License

This project is licensed under the MIT License. See LICENSE file for details.

Support & Contact

Author: AGBOKOUDJO Franck
Email: franckagbokoudjo301@gmail.com
Phone: +229 0167 25 18 86
LinkedIn: Internationales Web Services
Company: INTERNATIONALES WEB SERVICES

For questions, suggestions, or issues, please reach out to the author or open an issue on the repository.

Last Updated: December 2024
Version: 1.0.0